

Titelblatt

Eidesstaatliche Erklärung

Danksagung?

Zusammenfassung

Abstract (englisch)

Inhaltsverzeichnis

A bkürzungsverzeichnis

1. Einleitung

Das Internet der Dinge (Internet of Things, IoT) vernetzt eine wachsende Anzahl an Geräten und Sensoren im Alltag und in der Industrie. Es sorgt dafür, dass Menschen und Dinge jederzeit und überall mit alles und jedem verbunden sein können. Viele dieser Geräte arbeiten jedoch an der sogenannten Netzwerkkante (Network Edge) und sind an physischen und ökonomischen Limitierungen gebunden. Sie verfügen meistens nur über eine geringe Rechenleistung, sehr begrenzten Speicherplatz und sind oft auf Batterien, Akkus oder einer ähnlichen Energieversorgung angewiesen. **Solche Geräte sind allgemeint bekannt als eingebettete Systeme (Ebedded Systems).** (Paper)

Diese eingebetteten Systeme sind zudem auch durch zeitliche Anforderungen charakterisiert. Um mit komplexen Aufgaben, wie die Erfassung von Sensordaten und drahtlose Kommunikation umgehen zu können, ist daher eine reine Programmierung ohne Betriebssystem (Bare-Metal-Programming) nicht ausreichend. Auf der anderen Seite sind vollwertige Standard-Betriebssysteme (Operating Systems, OS) wie Linux zu ressourcenintensiv für die Hardware. (Paper)

Hier kommen Echtzeitbetriebssysteme (Real-Time Operating Systems, RTOS) ins Spiel. Diese bieten eine effiziente Aufgabenverwaltung und **garantieren** vorhersagbare (deterministische) Reaktionszeiten für zeitkritische Prozesse. Ein quelloffener Vertreter ist das Zephyr RTOS, welches speziell für ressourcenbeschränkte, vernetzte eingebettete Systeme (Embedded Systems) konzipiert wurde und über eine modulare Architektur verfügt. (Paper)

1.1 Motivation

Die Skalierbarkeit und Konfigurierbarkeit von modernen OS wie Zephyr bieten Softwareentwicklern eine hohe Flexibilität. Auf der anderen Seite stehen sie jedoch vor einem Dilemma: Die Aktivierung von zusätzlichen Systemdiensten, Netzwerkprotokollen oder Schutzmechanismen kann den Speicherbedarf (RAM und Flash) erhöhen, wodurch die Echtzeitfähigkeit des Systems verändert werden kann.

Vor allem bei ressourcenbeschränkten Geräten können kleine Anpassungen in der Systemkonfiguration darüber entscheiden, ob eine Anwendung ihre vorgegebenen Fristen (Deadlines) einhält oder der physisch vorhandene Speicher überschritten wird. **Gerade im**

Kontext zu eingebetteten Systemen spielt der Determinismus eine wichtige Rolle, da das Verpassen der Deadlines zu Konsequenzen führen kann. Daher ist das zeitliche Verhalten eines RTOS wichtig.

Die Leistung eines RTOS hängt daher davon ab, wie die Parameter und die aktivierten Module gewählt sind. Daher ist es besonders wichtig in der Praxis, die genauen Auswirkungen und Einflüsse verschiedener Konfigurationen bzgl. der Systemleistung (empirisch) zu messen und die Ergebnisse zu verstehen.

1.2 Problemstellung und Forschungsfrage

Obwohl Zephyr ein weitverbreitetes RTOS ist und für eine Vielzahl von Hardware-Plattformen konzipiert ist, fehlen praktische Daten darüber, wie stark unterschiedliche Konfigurationsprofile die Hardware wirklich beeinflussen. Das Problem liegt darin, dass die Abhängigkeiten zwischen den aktivierten Softwaremodulen, wie bspw. Logging, und dem daraus entstehenden Ressourcenverbrauch auf realer Hardware nur schwer abschätzbar ist.

Aus dieser Herausforderung leiten sich der Titel der Arbeit, „*Leistungsbewertung eines Echtzeitbetriebssystems auf ressourcenbeschränkter Hardware*“ und die Forschungsfrage ab:

Wie verhalten sich Laufzeitverhalten und Speicherverbrauch von Zephyr RTOS in Abhängigkeit von der Konfiguration auf ressourcenbeschränkter Hardware?

Aus dieser Frage leiten sich zudem folgende Unterfragen ab:

1. *Wie verändert sich die Context-Switch-Latenz?*
2. *Wie verändert sich die Interrupt-Latenz?*
3. *Wie verändert sich das Timer-Jitter?*
4. *Wie beeinflussen die Konfigurationen Heap-Allokation und Speicherfragmentierung?*
5. *Wie beeinflussen die Konfigurationen den Speicherbedarf (RAM/ ROM)?*

1.3 Zielsetzung und Beitrag

Das Ziel dieser Arbeit ist es, ein systematisches und reproduzierbares Leistungsbewertungsverfahren (Benchmark) für das Zephyr RTOS auf einer ressourcenbeschränkten Zielplattform durchzuführen und zu evaluieren. Dabei werden drei verschiedene praxisnahe Konfigurationsszenarien definiert. Diese werden dann mit unterschiedlichen Benchmarks miteinander verglichen.

Die Konfigurationen umfassen:

1. **Minimal:** Das absolute Minimum an Funktionen und Features, die benötigt werden, um alle Benchmarks auf dem verwendeten Mikrocontroller durchführen und messen zu können.
2. **Logging:** Die Minimal-Konfiguration in Verbindung mit dem Logging-Feature, welches aktiviert wird. Diese Konfiguration dient dazu, den Einfluss eines gängigen Standard-features zu veranschaulichen.
3. **IoT-Stack:** Hierbei handelt es sich um eine IoT-Standard-Variante, in welcher ein Sensor seine Werte mittels CoAP-Server über DTLS bereitstellt und Firmware-Updates unterstützt.

Im Fokus der beschriebenen Untersuchung stehen zudem zwei Kernbereiche:

1. **Speicherverbrauch:** Die Messung, wie sich die verschiedenen Konfigurationen auf den statischen und dynamischen Speicherbedarf (RAM und ROM) auswirken.
2. **Laufzeitverhalten:** Die Analyse von Echtzeit-Metriken, wie der Dauer von Kontextwechseln (Context Switches) und der Reaktionszeit (Reaction Time) auf Hardware-Interrupts (Interrupt Latency).

Der Beitrag dieser Arbeit liegt darin, Metriken zu messen, um somit die Mechanismen des Systems anhand von empirischen Messdaten aufzuzeigen und zu evaluieren. Die daraus resultierenden Erkenntnisse sollen Entwicklern eine Hilfestellung und Entscheidungsgrundlage bieten, um Zephyr für zukünftige IoT-Anwendungen ressourcenschonend und optimal zu konfigurieren. Zudem sollen die gemessenen Ergebnisse eine Vergleichsgrundlage bieten, um Zephyr mit anderen RTOS, wie bspw. RIOT oder FreeRTOS vergleichen zu können. Dies soll Entwicklern helfen, sich für das passende OS basierend auf die Anforderungen ihrer Anwendung zu entscheiden.

1.4 Aufbau der Arbeit (Provisorisch)

Einleitung X

Motivation X

Problemstellung und Forschungsfrage X

Zielsetzung und Beitrag X

Grundlagen

Systeme und zeitliche Anforderungen X

Real Time Systems X

Embedded Systems X

Real Time Operating Systems

Zephyr RTOS

Verwandte Arbeiten/ State of the Art

Methodik & Versuchsaufbau X

Forschungsdesign X

Zielplattform X

Softwareumgebung X

Benchmark-Design

Context-Switch

Semaphore

Yield

Interrupt latency

Timer Jitter

Heap Allocation

Fragmentation

Implementierung

Projektstruktur

Benchmark Infrastruktur

Implementierung der Benchmarks

Konfigurationen

Minimal

Logging

IoT-Stack

Messergebnisse

Context-Switch

Interrupt latency

Timer Jitter

Heap Allocation

Fragmentation

Diskussion

Einfluss Logging

Einfluss IoT Stack

Trade Offs

Vergleich mit Literatur

Validität

Interne Validität

Externe Validität

Overhead

Limitierungen

Fazit

Ausblick

2. Grundlagen

Im folgenden Abschnitt werden grundlegende Begriffe für das Verständnis dieser Arbeit definiert. Dabei werden unter anderem die Begriffe *Job* und *Task* verwendet. Ein Job umfasst hierbei jede Einheit an Arbeit, welche vom System geplant und umgesetzt wird. Entsprechend wird eine Menge von Jobs als Task bezeichnet.

2.1 Systeme und zeitliche Anforderungen

System Ein System ist eine Abbildung bestehend aus einer Menge von Eingaben und Ausgaben (Inputs und Outputs). Die internen Details können dabei, sofern sie nicht von besonderem Interesse sind, als eine Art schwarze Box (Black Box) abstrahiert werden. (Laplane)

Release Time Der Freigabezeitpunkt (Release Time) eines Jobs bezeichnet den Zeitpunkt, in welcher ein Job zur Umsetzung zur Verfügung steht. Dieser Job kann ab dem Zeitpunkt seiner Release Time zu jeder Zeit geplant und umgesetzt werden. Ein Job hat keine Release Time, sobald alle Jobs zu dem Zeitpunkt der Umsetzung des Systems stattfinden. (Liu)

Completion Time Der Abschlusszeitpunkt (Completion Time) eines Jobs bezeichnet den Zeitpunkt, an welchem dieser fertig umgesetzt wurde.

Response Time Unter der Antwortzeit (Response Time) eines Systems versteht man die Zeit, zwischen dem Anlegen einer Menge von Inputs an ein System und der Realisierung des geforderten Verhaltens, inklusive der Verfügbarkeit aller zugehörigen Outputs. Wie schnell und zeitlich genau die Response Time sein muss, hängt von den Eigenschaften und dem Zweck des jeweiligen Systems ab. (Laplane) Des weiteren lässt sich die Response Time eines Jobs als die Zeitspanne von der Release Time bis zu dem Zeitpunkt, an dem er abgeschlossen ist, beschreiben. (Liu)

Deadline Die Deadline eines Jobs ist der Zeitpunkt, zu dem seine Ausführung abgeschlossen sein muss. Die Zeitliche Anforderung eines Jobs kann dabei in Form seiner Response Time beschrieben werden. Die maximal erlaubte Response Time eines Jobs wird daher als seine Relative Deadline bezeichnet. Addiert man die Relative Deadline mit der Release Time eines Jobs, so erhält man seine Absolute Deadline. Wird der Job spätestens zu seiner absoluten Deadline abgeschlossen, so gilt die zeitliche Anforderung als eingehalten. (Liu)

Tardiness Die Tardiness eines Jobs misst, wie sehr dieser von seiner Deadline abweicht. Die Tardiness ist Null, wenn der Job vor oder zu seiner Deadline abgeschlossen ist. Falls dies nicht der Fall ist, ist der Job zu spät und die Tardiness entspricht der Differenz zwischen seiner Completion Time und Deadline.

Timing Constraint Eine Einschränkung, die dem zeitlichen Verhalten eines Jobs auferlegt wird, wird allgemein als zeitliche Einschränkung (Timing Constraint) bezeichnet. Die Timing

Constraint eines Jobs kann beispielsweise anhand seiner Release Time und seiner relativen oder absoluten Deadline spezifiziert werden. Bei einigen komplexen Timing Constraints ist dies jedoch nicht der Fall. Timing Constraints können hierbei in zwei Kategorien aufgeteilt werden: Hart (Hard) und Weich (Soft). In dieser Untersuchung wird der Unterschied zwischen Hard und Soft Timing Constraints im Bezug zu ihrer Nützlichkeit für die System Performance unterschieden. Die Nützlichkeit eines Ergebnisses von einem Job, mit einer Soft Deadline nimmt allmählich mit dem Anstieg der Tardiness ab. Auf der anderen Seite nimmt die Nützlichkeit des Ergebnisses eines Jobs mit einer Hard Deadline abrupt ab, oder wird sogar negativ, sobald die Tardiness größer als Null ist. (Liu)

Failed System Ein ausgefallenes System (Failed System) ist ein System, das eine oder mehrere Anforderungen, welche in den Systemspezifikationen festgelegt sind, nicht erfüllen kann. (Laplante)

2.2 Real Time Systems

Real Time

Real-Time System Die vorherigen Definitionen bilden die Grundlage für eine praktische Definition eines Echtzeitsystems (Real-Time System, RTS). Ein RTS ist ein Computersystem, dessen logische Korrektheit sowohl auf der Korrektheit der Outputs als auch auf deren Rechtzeitigkeit beruht. In jedem Fall wird durch das Weglassen der Notwendigkeit des Konzepts der Rechtzeitigkeit jedes System zu einem RTS. Echtzeitsysteme sind oft reaktive oder eingebettete Systeme. Reaktive Systeme sind dabei welche, bei denen die Aufgabenplanung durch die fortlaufende Interaktion mit ihrer Umgebung gesteuert wird. RTS müssen zeitlich begrenzte Response Time Bedingungen erfüllen, da ansonsten schwerwiegende Konsequenzen inklusive eines System Ausfalls (System Failure) riskiert werden. (Laplante)

Soft RTS Ein Soft RTS ist ein System, bei dem die Leistungsfähigkeit beeinträchtigt, aber nicht vollständig zerstört wird, wenn Response-Time Bedingungen nicht eingehalten werden. (Laplante)

Hard RTS Ein Hard RTS ist ein System, bei dem die Nichteinhaltung von nur einer einzigen Deadline zu einem vollständigen oder katastrophalen System Failure führen kann. (Laplante)

Firm RTS Ein Firm RTS ist ein System, bei dem das Verpassen einiger weniger Deadlines nicht zu einem vollständigen System Failure führt, das Verpassen von mehr als einigen wenigen Deadlines jedoch zu einem vollständigen oder katastrophalen System Failure führen kann. (Laplante)

2.3 Embedded Systems

Embedded Systems Ein eingebettetes System ist ein System, das einen oder mehrere Computer, oder Prozessoren, enthält, die eine zentrale Rolle für die Funktionalität des Systems spielen, wobei jedoch das Gesamtsystem nicht notwendigerweise als Computer bezeichnet wird. (Laplante) Eingebettete Systeme werden typischerweise für die Erfüllung von einer oder einer begrenzten Menge an Funktionen mithilfe von Hardware und Software entwickelt. Hardware und Software machen ein eingebettetes System zu einem Computer. Der Unterschied zu herkömmlichen PCs ist jedoch, dass eingebettete Systeme nur eine begrenzte Menge an Funktionen erfüllen können. Laut Lyla B. Das gehören PCs somit zu der Kategorie „General Purpose Computer“, während eingebettete Systeme zu „Special Purpose Computer“ gehören. Die für die Funktion des eingebetteten Systems erforderliche Software

wird dabei als „Firmware“ bezeichnet. Allgemein betrachtet besteht ein eingebettetes System aus einem Prozessor, Sensoren, Aktoren und Zwischenspeicher. Eine Anwendung soll also mit den Sensoren Daten einlesen können, welche vom Prozessor verarbeitet und an die Aktoren weitergegeben werden, um eine bestimmte Aufgabe zu erfüllen. (Das) Viele eingebettete Systeme sind Hard RTS. Jedoch können diese je nach Anwendung unterschiedliche Anforderungen aufweisen. Die Deadlines von Jobs in einem eingebetteten System werden aus der Reaktionsfähigkeit der Sensoren und Aktoren abgeleitet, welche von diesem überwacht und kontrolliert werden. (Liu)

Mikrocontroller Ein Mikrocontroller (Microcontroller Unit, MCU) verfügt nicht nur über einen Prozessor, sondern auch über timer, parallele und serielle Schnittstellen, RAM, ROM, etc. Programm Code wird auf den internen ROM gebrannt und Anwendungs-Code läuft innerhalb des internen RAMs. Ein Mikrocontroller ist somit ein eigener Single-Chip Computer. Wenn ein solcher Mikrocontroller über eingebaute Peripheriegeräte verfügt, sodass das Design eines großen Systems mit diesen möglich ist, dann wird dieser Mikrocontroller ein „System on Chip“ (SoC) genannt.

2.4 Real Time Operating Systems

2.5 Zephyr RTOS

Zephyr ist ein quelloffenes (open-source) RTOS, das für den Einsatz auf ressourcenbeschränkten eingebetteten Systemen entwickelt wurde. Das Projekt wird von der Linux Foundation als ein gemeinschaftlich entwickeltes Open-Source-Projekt unterstützt. Zephyr verfolgt das Ziel, ein skalierbares und plattformübergreifendes OS für unterschiedliche Embedded-Anwendungen bereitzustellen. Zephyr kann, durch die Unterstützung unterschiedlicher Prozessorarchitekturen und Hardwareplattformen auf Systemen mit verschiedenen Ressourcen- und Leistungsanforderungen eingesetzt werden. Die Architektur ist modular und konfigurierbar aufgebaut. Dadurch können benötigte Komponenten deaktiviert werden. Der Zephyr-Kernel unterstützt zudem kooperatives und präemptives Threading und stellt Funktionen für die Aufgabenverwaltung, Synchronisation, Speicherverwaltung und Interaktion mit Hardware bereit. Außerdem verfügt Zephyr über integrierte Schnittstellen für Gerätetreiber und Subsysteme für bspw. Metzerkommunikation, Bluetooth, USB, Dateisysteme und Logging. Durch die Möglichkeit, Zephyr zu konfigurieren, kann der benötigte Speicher- und Funktionsumfang an die verfügbare Hardware angepasst werden. Auf der anderen Seite unterstützt Zephyr auch leistungsfähigere Systeme. (Zephyr About the Zephyr Project)

Zephyr basiert auf einem Kernel mit geringem Speicherbedarf. Dieser wurde für ressourcenbeschränkte und eingebettete Systeme entwickelt. Der Zephyr Kernel unterstützt hierbei eine Vielzahl von CPU-Architekturen, darunter die in dieser Arbeit verwendete ARMv7-M (Cortex-M4) Architektur. Zephyr wurde zudem durch Subsysteme modular konzipiert. Subsysteme umfassen hierbei logisch abgegrenzte Teile des OS, welche bestimmte Dienste bereitstellen, darunter beispielsweise Netzwerke, Dateisysteme oder Kommunikationsprotokolle, etc. Jedes Subsystem ist modular aufgebaut und kann somit konfiguriert, angepasst und erweitert werden. (Zephyr Doku Einleitung)

Zephyr ist hauptsächlich in C geschrieben und unterstützt daher Anwendung, die in C geschrieben sind. Zephyr verfügt über einige API-Funktionen und API-Makros, welche durch C-Header-Dateien inkludiert werden können. Dabei muss die Funktion „main()“ den

Rückgabewert int (integer) haben und den Wert Null (0) zurückgeben. Zephyr unterstützt auch weitere Sprachen wie C++ und Rust, wobei diese für die vorliegenden Untersuchungen nicht von Relevanz sind. (Zephyr Doku Language Support)

3.Methodik & Versuchsaufbau

Forschungsdesign

Die Arbeit verfolgt ein quantitatives und experimentelles Forschungsdesign mit kontrollierter Variation der Zephyr-Konfiguration. Dabei werden wiederholte Messungen auf derselben Hardwareplattform durchgeführt (Within-Subject-Design). Dieselbe physische Plattform, ein nrf52840-DK, durchläuft nacheinander drei Firmware-Builds, die sich jeweils in der zu untersuchenden Kernel-/ Stack-Konfiguration unterscheiden. Alle übrigen relevanten Rahmenbedingungen, darunter Hardware, CPU-Taktfrequenz, Zephyr-Version, Compiler und Messverfahren, werden dabei konstant gehalten. Dadurch werden hardwarebedingte Unterschiede zwischen den Messungen minimiert. Zudem können Unterschiede in den Messergebnissen auf die Konfigurationsänderungen zurückgeführt werden, soweit andere Einflussfaktoren kontrolliert werden (Ceteris-Paribus-Prinzip). Quantitativ bedeutet im Kontext dieser Untersuchung, dass diese anhand von messbaren numerischen Größen durchgeführt wird. Zudem bedeutet experimentell, dass Einflussfaktoren gezielt verändert werden und beobachtet wird, wie sich dadurch eine abhängige Messgröße verändert.

Das Versuchsdesign folgt somit der klassischen Struktur eines kontrollierten Experiments:

- **Unabhängige Variable (UV):** Die aktive Zephyr-Konfiguration (Minimal, Logging, IoT-Stack) ist der zu manipulierende Faktor.
- **Abhängige Variablen (AV):** Die fünf gemessenen RTOS-Leistungsmetriken (Context-Switch, Interrupt Latenz, Timer-Jitter, Heap-Allokationsgeschwindigkeit, Heap-Fragmentierung), wobei hierbei die Wirkung der Zephyr-Konfigurationen beobachtet wird.
- **Kontrollierte Variablen:** Alle für die Messung relevanten Rahmenbedingungen, die zwischen den Durchläufen konstant gehalten werden, um Reproduzierbarkeit zu ermöglichen. Dazu gehören die verwendete Hardware, CPU-Taktfrequenz, Compiler, Zephyr-Version, Messverfahren, Stichprobenumfang und Umgebung. Die Anzahl der Messwiederholungen und der Messmechanismus für die Datenerhebung für alle Konfigurationen sind ebenfalls festgelegt.

Jede Kombination aus Konfiguration und Metrik (In der restlichen Arbeit als „Zelle“ bezeichnet) wird als eigenständiges, minimales Firmware-Image gebaut. Dabei wählt CMakeLists.txt durch die Methode `target_sources_ifdef` genau eine `bm_*.c`-Datei zur Build-zeit aus. `Kconfig` macht darüber hinaus die Auswahl über `choice BENCH_TYPE` explizit. Durch diese Isolation wird verhindert, dass sich mehrere Benchmarks im selben Binary gegenseitig durch Codegrößen-, Cache- oder Speicherlayout-Effekte verfälschen.

Die drei Konfigurationsstufen sind zudem kumulativ/ inkrementell angelegt. Die Datei `logging.conf` ist hierbei lediglich die `minimal.conf` inklusive eines Features. Dadurch wird nicht nur das IoT-Stack mit dem Kernel-Build verglichen, sondern auch der isolierte Effekt einzelner Subsysteme, in diesem Fall das Logging-Subsystem sauber quantifizierbar bleibt. Durch diese Methodik wird eine Stufenanalyse ermöglicht anstelle eines Zwei-Gruppen-Vergleichs.

Für jede Zelle des Versuchsplans wird eine Stichprobe von N Messwerten erhoben, aus der deskriptive Kennzahlen berechnet werden. Diese sind das Minimum, Maximum, der Mittelwert und die Stichproben-Standardabweichung. Außerdem existiert ein Referenzpunkt gegen Zephyr's eigene Benchmarks, welche dieselbe Timing-API verwenden. Dies dient als externe Validierung der selbst entwickelten Messungen.

Unabhängige Variablen

Die UV werden als Kconfig-Fragmente realisiert, das über `EXTRA_CONF_FILE` zusätzlich zur minimalen Basis `prj.conf` der Mess-App eingebunden wird. Die Basis enthält nur das, was für jede Messung zwingend nötig ist, darunter die Timing-API, UART/ printk-Ausgabe und das heap für die Allocation-Benchmarks. Diese Basis ist bewusst identisch in allen drei Konfigurationsstufen, damit diese nicht Teil der Variation wird.

1 Minimal Konfiguration

Der schlanke RTOS-Kern ohne jegliche Zusatzdienste. Boot-Banner, Timeslicing, Assertions, Logging, Thread-Namen, MPU/ Stack-Guard, SEGGER-RTT und GPIO-Treiber sind explizit deaktiviert. Diese Konfigurationsstufe dient als Baseline und liefert die „reinen“ Kernelkosten ohne fremdeinflüsse. Besonders ohne MPU-bedingte Stack-Guard-Reprogrammierung bei jedem Context Switch und ohne durch Timeslicing verursachte Scheduler-Unterbrechungen.

2 Logging

Diese Konfiguration baut auf der Minimal-Konfiguration auf und fügt genau ein feature-Delta hinzu: Das Zephyr-Logging-Subsystem im Deferred-Modus. Dies beinhaltet einen Ringpuffer und einen eigenen Log-Thread HIER UART. Diese Stufe isoliert die Kosten eines einzelnen Dev-/ Diagnose-Features und beantwortet die Frage, welche Kosten ein einzelnes Feature wie Logging in Anbetracht der Laufzeit und dem Footprint verursacht.

3 IoT-Stack

Dies ist die komplexeste Stufe.

HIER IOT STACK ERKLÄREN SOBALD FERTIG.

Abhängige Variablen

Alle Metriken sind in eigenen `bm_*.c` Dateien unter `src` implementiert, folgen einem einheitlichen Muster und werden in `METHODIK.md` mit Definitionen, Messpunkten und Fehlerquellen dokumentiert. Das Muster legt einen Zeitstempel vor und nach dem Ereignis via `bench_now()`, eine Overhead-Korrektur, Akkumulation in `struct bench_stats` und einer CSV-Ausgabe via `bench_report()` fest.

1 Context-Switch-Latenz

Der Context-Switch wurde im Benchmark `bm_ctx_switch.c` implementiert. Hier wird die Zeit vom Auslösen eines Threadwechsels bis zur ersten Instruktion im Zielthread gemessen. Dies erfolgt in zwei Varianten:

`ctx_switch_sem_wakeup`: Semaphore-geweckter höherpriorer Worker-Thread

`ctx_switch_yield`: Kooperatives Ping-Pong gleichpriorer Threads via `k_yield()`.

Dieser Benchmark erfasst die Scheduler-Entscheidung inklusive PendSV-Kontextwechsel. Grundkosten jeder Multithreading-Architektur, zentrale Vergleichsgröße zwischen den drei Konfigurationen.

2 Interrupt-Latenz

Die Interrupt-Latenz wurde im Benchmark `bm_irq_latency.c` implementiert. Es wird die Zeit vom Software-Trigger bis zur ersten ISR-Instruktion gemessen. Dies erfolgt durch `NVIC_SendPendingIRQ` auf einer ungenutzten Peripherie-leitung des nrf52840. Dieser benchmark zeigt, wie stark Kernel-Features, wie bspw. Logging, den determinismus-kritischsten Pfad eines RTOS beeinflussen.

3 Timer-Jitter

Der Timer-Jitter wurde im Benchmark `bm_timer_jitter.c` implementiert. Hier wird die Abweichung der Ist- von der Soll-Periode eines periodischen 10 ms `k_timer` gemessen. Die Messung läuft über `k_uptime_ticks()`, was RTC-1 gestützt ist und dadurch auch im Schlaf läuft.

4 Heap-Allokationsgeschwindigkeit

Die Heap-Allokationsgeschwindigkeit wurde im Benchmark `bm_heap_alloc.c` implementiert. Hier wird die Zeit für die Funktionen `k_malloc` und `k_free` eines 64-Byte-Blocks gemessen. Dabei sind die Alloc und Free Befehledirekt aufeinanderfolgend, wobei der Best Case ein leerer Heap darstellt. Dieser benchmark ist relevant, weil `sys_heap` eine variable Laufzeit hat, was für RTS kritisch ist.

5 Heap-Fragmentierung

Die Heap-Fragmentierung wurde im Benchmark `bm_heap_frag.c` implementiert. Hierbei handelt es sich nicht um ein Zeit-, sondern um ein Zustandsmaß. Ein eigener 8-KiB `k_heap` wird mit 32x128-Byte-Blöcken gefüllt. Dabei wird im Schachbrettmuster jeder zweite freigegeben und anschließend die Belegung und der größte zusammenhängende Freiblock protokolliert. Dieser freiblock wird durch eine eigene Bisektions-Probe protokolliert, da `sys_heap` diese Kennzahl nicht direkt liefert. Dieser Benchmark zeigt ein für MMU-lose Systeme spezifisches Risiko: Rechnerisch ausreichender, aber durch Fragmentierung nicht mehr am Stück nutzbarer Speicher.

Alle Metriken, abgesehen von der Heap-Fragmentierung, liefern Zeitwerte in CPU-Zyklen und werden identisch statistisch ausgewertet. Die Heap-Fragmentierung hingegen liefert Bytes bzw. Zustände und wird daher mit einem separaten Ausgabeformat ausgegeben, um sie nicht fälschlicherweise in dieselbe Zeit-Statistik einzumsuchen. Dies erfolgt in `HEAPSTATS` und `HEAPPROBE` anstatt der `BENCH-CSV`.

Kontrollierte Variablen

Diese Variablen werden püber alle Konfigurationsstufen und Metriken hinweg konstant gehalten, um zu verhindern, dass Effeekte fälschlich der UV zugeschrieben werden.

Hardware/ CPU-Frequenz

Ein einziges physisches Board, das Nordic nrf52840 DK, mit einer CPU basierend auf der Cortex-M4F Architektur und einer Takt-Frequenz von 64 MHz. Es verfügt über 1 MB Flash und 256 KB RAM. Die Frequenz wird darüber hinaus zur Laufzeitkontrolle bei jedem Boot über die Methode `timing_freq_get()` in der `BENCHMETA` zeile in `main.c` mit ausgegeben.

Falls es also zu einer Abweichung in der Takt-frequenz kommen sollte, wäre dies sofort sichtbar.

Compiler und Toolchain

In dieser Untersuchung wurde das Zephyr Software Development Kit (SDK) 0.16.9 verwendet. Der Ccompiler ist daher GCC 12.2.0, mit Binutils 2.38 und der Picolibc 1.8.6. Diese wurden über ZEPHYR_SDK_INSTALL_DIR gepinnt, sodass sie identisch für alle Builds sind.

Compiler optimierung

Es wurde der Zephyr Default -Os in SIZE_OPTIMIZATIONS für alle drei konfigurationen festgelegt. Keine der Konfigurationsstufen ändert das Optimierungslevel.

Zephyr Version

Bei der verwendeten Zephyr Version handelt es sich um die Latest Stable Version (LTS), welches zum Zeitpunkt des Schreibens dieser Arbeit Zephyr 3.7.2 LTS ist. Dies wurde im Manifest west.yml über revision: 3.7.2 mit import: true gepinnt. Dies übernimmt passende HAL-, MCUBoot-, mbedTLS- und OpenThread-Revisionen. Damit ist ausgeschlossen, dass ein Versionsdrift einen Einfluss zwischen den drei Messläufen nimmt.

Build-Konfiguration (Sockel)

Die in prj.conf definierte basis ist für alle drei UV-Stufen identisch. Diese beinhaltet **Benchlib**, **printf/ UART/ Konsole**, **CONFIG_CBPRINTF_FP_SUPPORT** für Fließkomma-Ausgabe der Statistik und einen 64-KiB-System-Heap. Diese Basis wird nicht durch die Konfigurationen verändert, sondern lediglich ergänzt. Dadurch bleibt bspw. der ROM-Overhead der Fließkomma-Ausgabe in jeder Konfiguration gleich groß und ermöglicht somit einen Vergleich.

Messmechanismus (Timing/ Statistik)

Der in **CONFIG_TIMING_FUNCTIONS** implementierte Messmechanismus ist für alle Zeitmetriken identisch. Dabei wird über den DWT-Zyklenzähler gemessen (DWT->CYCCNT, 1 Tick = 1 Takt = 15,625 ns @ 64 MHz). Zudem wird der Messoverhead durch zwei aufeinanderfolgende Zähler-Reads kalibriert mit einem Minimum von 100 Versuchen und anschließend von jedem Ergebnis subtrahiert. Dieser Mechanismus wurde in **bench_timing.c** implementiert.

Die Statistik basiert auf den Welford-Algorithmus, implementiert in **bench_stats.c**. Dabei handelt es sich um eine inkrementelle, numerisch stabile Berechnung von Mittelwert und Varianz ohne Speicherung der Einzelwerte (n-1-Stichproben-Standardabweichung) inklusive eines laufenden Minimums und Maximums. Diese identische Mess- und Auswertungskette über alle Konfigurationen und Metriken hinweg sorgt dafür, dass beobachtete Unterschiede nicht auf unterschiedliche Messverfahren zurückgehen. Die Einzige Ausnahme bildet der Timer-Jitter-Benchmark, welcher **k_uptime_ticks()** verwendet anstelle des DWT. Aber auch dieser Unterschied wird über alle Konfigurationen hinweg verwendet und sorgt somit für keine Inkonsistenz.

Anzahl der Messungen

Die Anzahl der Messungen wurde ebenfalls einheitlich gehalten: N = 1000 gewertete Iterationen, nachdem 10 Warm-up-Läufe verworfen wurden, da es bei den ersten Iterationen

zu Cache-/ Pipeline- und Startpfad-Effekten kommen könnte, welche die Ergebnisse verfälschen würden. Dies ist ebenfalls konfigurierbar über `CONFIG_BENCHLIB_ITERATIONS` und `CONFIG_BENCHLIB_WARMUP` und wird zudem bei jedem Lauf in der **BENCHMARK**-Kopfzeile protokolliert. Dadurch ist die Stichprobengröße jeder Messreihe aus den Rohdaten nachvollziehbar und über alle Zellen des Versuchsplans identisch.

Zielformat

Als Zielformat für die Untersuchungen wird das nRF52840 Development Kit von Nordic Semiconductor verwendet. Die Plattform basiert auf einem 32-Bit Arm Cortex-M4 mit einer Taktfrequenz von 64 MHz und verfügt über 1 MB Flash-Speicher sowie 256 kB RAM. Damit bietet der Mikrocontroller eine klar definierte und ressourcenbeschränkte Hardwarebasis für die Untersuchung des Laufzeit- und Speicherverhaltens von Zephyr RTOS.

Für die Untersuchung der Laufzeiteigenschaften sind vor Allem der Nested Vectored Interrupt Controller (NVIC) und der Data Watchpoint and Trace (DWT) des Cortex-M4 relevant. Der NVIC übernimmt die Verarbeitung und Priorisierung von Interrupts und bildet damit die hardwareseitige Grundlage für die Messung der Interrupt-Latenz. Der DWT ermöglicht die Erfassung von CPU-Taktzyklen und wird für die hochauflösende Laufzeitmessung der Benchmarks eingesetzt. Bei 64 MHz entspricht ein CPU-Zyklus 15,625 ns, wodurch beispielsweise Kontextwechsel- und Interrupt-Laufzeiten mit hoher zeitlicher Auflösung bestimmt werden können.

Für die Untersuchung des Timer-Jitters wird auf der anderen Seite die RTC-basierte Zeitbasis von Zephyr verwendet. In der verwendeten Konfiguration wird hierfür RTC1 mit einem 32,768-kHz-Low-Frequency-Clock betrieben. Da diese Zeitbasis auch während des Prozessorschlafs weiterläuft, eignet sie sich zur Messung periodischer Timerintervalle im Tickless-Betrieb. Die resultierende Auflösung beträgt etwa 30,5 µs pro Tick. Die Kombination aus Cortex-M4, Interrupt- und Zeitgeberhardware sowie den begrenzten Speicherressourcen bildet damit die technische Grundlage für die durchgeführten Benchmarks. ([Nordic Semiconductors Doku](#))

Softwareumgebung

Das Build befindet sich auf demselben Ubuntu-Host im Workspace `~/zephyr-ws`. Dabei handelt es sich um eine eigene Python-venv (Virtual Environment). Das Flashen auf den Mikrocontroller erfolgt über denselben J-Link-OB-Debugprobe via west flash (nrfjprog-Runner). Die Ausgabe erfolgt dann über denselben UART/ VCOM-Kanal. Pro Messung werden ebenfalls der Commit-Hash und das verwendete **EXTRA_CONF_FILE** notiert.

Abkürzungen

Quellen

Überschriften

Unterpunkte

Eigene Aussagen (Temporär bis Quelle gefunden)

Dateien

Grundlagen:

Liu: s.26-31 RTS & Embedded Systems

Das: s.34-39 Embedded Systems& MCU

Das: s.47-49 MCU

Benchmarks:

Das: s.52-X Definitionen für ram, rom, interrupts etc.

Methodik Erweiterung (Noch nicht fertig, aus Zephyr Doku, einige davon wurden verwendet)

1 Optimierungen

Zephyr lässt sich hinsichtlich der Leistung, dem Energieverbrauch und dem Speicherbedarf (Footprint) optimieren.

Optimierung des Speicherbedarfs

Stack Größen

Die Stäck-Größen der verschiedenen Systemthreads lassen sich für jede Anwendung anpassen. Dabei sind folgende Standardwerte gesetzt:

CONFIG_ISR_STACK_SIZE: 2048

CONFIG_MAIN_STACK_SIZE: 1024

CONFIG_IDLE_STACK_SIZE: 320

CONFIG_SYSTEM_WORKQUEUE_STACK_SIZE: 1024

CONFIG_PRIVILEGED_STACK_SIZE: 1024

Peripheriegeräte

Einige Peripheriegeräte sind standardmäßig aktiviert. Diese lassen sich ebenfalls folgendermaßen deaktivieren:

CONFIG_GPIO=n

CONFIG_SPI=n

Debug-/ Informationsoptionen

Die untenstehenden Optionen bieten die Möglichkeit, mehr Informationen über die laufende Anwendung auszugeben und dienen primär als Mittel für Debugging und Fehlerbehandlung (Error-Handling):

CONFIG_BOOT_BANNER

CONFIG_DEBUG

Der Boot Banner ist hierbei standardmäßig aktiviert und kann einige Bytes in Anspruch nehmen.

MPU/ MMU-Unterstützung

Um weiterhin Speicher zu sparen und die Leistung zu verbessern, kann die MPU/ MMU-Unterstützung deaktiviert werden. Dadurch geht jedoch die erweiterte Stack-Überprüfung und -Unterstützung verloren.

2 Optimierungswerkzeuge

Zephyr verfügt über einige Werkzeuge, welche die Möglichkeit bieten, den Footprint, die Speichernutzung und Datenstrukturen über verschiedene Build-System-Ziele (Targets) analysieren zu können.

2.1 Footprint und Speichernutzung

Für die Ansicht und Analyse von RAM-, ROM-, und der Stack-Nutzung in erzeugten Images bietet Zephyr zudem drei unterschiedliche Targets. Die Werkzeuge laufen demnach auf dem final erzeugten Image und stellen Informationen über die Größe von Symbolen und Code dar, welche sowohl im RAM als auch im ROM verwendet werden. Außerdem kann mithilfe von Compiler Funktionen auch eine Worst-Case-Stack-Nutzungsanalyse erzeugen. Diese Werkzeuge wurden in dieser Arbeit verwendet:

2.1.1 ram_report

Alle kompilierten Objekte und ihre RAM-Nutzung werden in tabellarischer Form aufgelistet. Aus dieser Tabelle kann man Informationen wie Bytes pro Symbol und dem prozentualen Anteil einsehen. Diese Daten werden basierend auf dem Speicherort des Objekts im Dateisystembaum und der Datei gruppiert, die das Symbol enthält. Dies lässt sich mit folgenden West Befehlen umsetzen:

```
west build -b boardname filename
```

```
west build -t ram_report
```

2.1.2 rom_report

Alle kompilierten Objekte und ihre ROM-Nutzung werden in tabellarischer Form aufgelistet. Aus dieser Tabelle kann man Informationen wie Bytes pro Symbol und dem prozentualen Anteil einsehen. Diese Daten werden basierend auf dem Speicherort des Objekts im Dateisystembaum und der Datei gruppiert, die das Symbol enthält. Dies lässt sich mit folgenden West Befehlen umsetzen:

```
west build -b boardname filename
```

```
west build -t rom_report
```

2.1.3 ram_plot/ rom_plot

Zusätzlich zu den Targets `ram_report` und `rom_report` erzeugen `ram_plot` und `rom_plot` Speichernutzungsberichte, aber in Form von Diagrammen. Die Segmente sind interaktiv und hilft dabei, durch die Verzeichnisstruktur zu navigieren und weitere Details auslesen zu können. Sobald der folgende Befehl ausgeführt wird, wird zuerst ein CLI-Bericht erstellt und anschließend ein Browser-Fenster geöffnet:

```
west build -b boardname filename
```

```
west build -t ram_plot oder rom_plot
```

2.1.4 Dashboard

Durch dieses Werkzeug lässt sich ein HTML-Dashboard erzeugen, welches die verschiedenen Werkzeugausgaben und Artefakte in einer einzigen Ansicht zusammenfasst. Neben der Zusammenfassung der Build-Ergebnisse sind zusätzlich vollständige Speicherberichte zu RAM und ROM in Form von Tabellen und Diagrammen enthalten. Kconfig-Symbolwerte und deren Quellen werden ebenfalls generiert. Zudem wird eine navigierbare Device-Tree-Ansicht erzeugt, in welcher weitere Details eingesehen werden können. Dies wird durch folgenden Befehl ermöglicht:

```
west build -b boardname filename
```

```
west build -t dashboard
```

3 Flashing

Zephyr - Grundlagen

- 1 Kernel
- 2 Threading
- 3 Scheduling
- 4 Synchronization
- 5 Interrupts
- 6 Timing
- 7 Memory

Messgrundlagen

- 1 Logging
- 2 Tracing
- 3 Instrumentation
- 4 CPU load

Konfiguration

- 1 CMake
- 2 Devicetree
- 3 Kconfig
- 4 Snippets
- 5 Flashing
- 6 Sysbuild?

Hardware – Cortex M4F

- 1 GPIO
- 2 Timer
- 3 UART
- 4 Bluetooth
- 5 SPI/ I2C

<https://www.zephyrproject.org/learn-about/>

<https://thesis.dial.uclouvain.be/server/api/core/bitstreams/c6169167-70c9-42c0-9654-c01f87638fb0/content>

<https://link.springer.com/book/10.1007/978-1-4614-3143-5>

<https://link.springer.com/book/10.1007/978-3-031-28701-5>

<https://link.springer.com/book/10.1007/978-3-031-73098-6>

<https://www.intel.com/content/www/us/en/developer/articles/community/zephyr-story-how-became-self-sustaining-ecosystem.html>